

pandas 데이터 재구조화(reshaping)

- 피벗팅(pivoting)
- 스택킹(stacking)과 언스택킹(unstacking)
- 멜팅(melting)과 와이드투롱(wide_to_long)
- 교차표(crosstab)
- explode

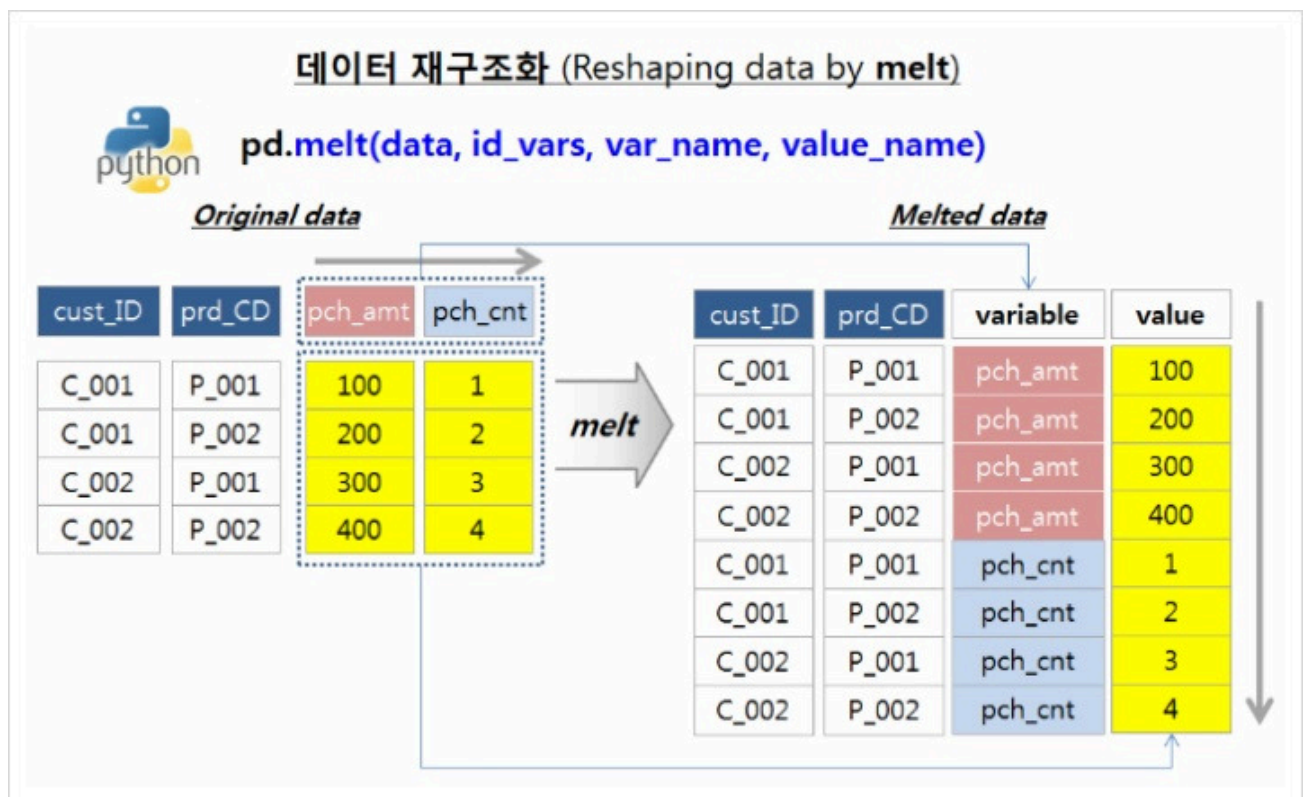
```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}, Seaborn:{sns.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5, Seaborn:0.13.2

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

3. 멜팅(melting)

- 식별자 변수로 사용될 id를 기준으로 원래 데이터셋에 있던 여러 개의 컬럼 이름을 'variable' 컬럼에 위에서 아래로 길게 쌓아놓고, 'value' 컬럼에 id와 variable에 해당하는 값을 넣어주는 방식으로 데이터를 재구조화



- 출처 : <https://rfriend.tistory.com/278>

melt()

DataFrame.melt(id_vars=None, value_vars=None, var_name=None, value_name='value', col_level=None, ignore_index=True)

- id_vars : identifier variables로 사용될 컬럼(들)
- value_vars : unpivot할 컬럼(들). 지정하지 않는 경우 id_vars에 지정하지 않은 모든 컬럼(들)이 됨
- var_name : 기본값은 'None'. 'variable' 컬럼의 이름, 지정하지 않는 경우 frame.columns.name 또는 'variable'
- value_name : 기본값은 'value'. 'value'컬럼의 이름
- col_level : 컬럼들이 멀티인덱스인 경우 melt할 레벨
- ignore_index : 기본값은 True. True인 경우 원래 인덱스 무시하나 False인 경우 원래 인덱스 유지함

<https://pandas.pydata.org/docs/reference/api/pandas.melt.html>

Melt

df3					df3.melt(id_vars=['first', 'last'])				
	first	last	height	weight		first	last	variable	value
0	John	Doe	5.5	130	0	John	Doe	height	5.5
1	Mary	Bo	6.0	150	1	Mary	Bo	height	6.0
					2	John	Doe	weight	130
					3	Mary	Bo	weight	150

예제 데이터1

```
In [3]: df1 = pd.DataFrame({'first':['John','Mary'],  
                           'last':['Doe','Bo'],  
                           'height':[5.5,6.0],  
                           'weight':[130,150]})  
df1
```

```
Out[3]:
```

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150

- melt() : 모든 컬럼명이 variable의 값이 되고, 컬럼의 값은 value로

```
In [4]: df1.melt()
```

Out[4]:

	variable	value
0	first	John
1	first	Mary
2	last	Doe
3	last	Bo
4	height	5.5
5	height	6.0
6	weight	130
7	weight	150

- `melt(id_vars=[ ])`
 - `id_vars`에 지정된 컬럼은 그대로 유지(기준 컬럼)
 - 그외 컬럼이 녹여져 `variable`과 `value` 컬럼으로 변경

In [5]: `df1.melt(id_vars=['first', 'last'])`

Out[5]:

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

In [6]: `df1.melt(id_vars=['first'])`

Out[6]:

	first	variable	value
0	John	last	Doe
1	Mary	last	Bo
2	John	height	5.5
3	Mary	height	6.0
4	John	weight	130
5	Mary	weight	150

- `melt(id_vars=[ ], var_name=)`

In [7]: `df1.melt(id_vars=['first', 'last'], var_name='body')`

```
Out[7]:
```

	first	last	body	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

- `melt(id_vars=[ ], var_name= , value_name= )`

```
In [8]: df1.melt(id_vars=['first','last'], var_name='body', value_name='data')
```

```
Out[8]:
```

	first	last	body	data
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

- `melt(id_vars=[ ], var_name= , value_name=, value_vars= )`

```
In [9]: df1.melt(id_vars=['first','last'], value_vars=['height'])
```

```
Out[9]:
```

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0

```
In [10]: df1.melt(id_vars=['first'], value_vars=['height'])
```

```
Out[10]:
```

	first	variable	value
0	John	height	5.5
1	Mary	height	6.0

```
In [11]: df1.melt(id_vars=['first'], value_vars=['height','weight'])
df1.melt(id_vars=['first','last'], value_vars=['height','weight'])
```

```
Out[11]:
```

	first	variable	value
0	John	height	5.5
1	Mary	height	6.0
2	John	weight	130.0
3	Mary	weight	150.0

```
Out[11]:
```

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

```
In [12]: df1.melt(id_vars=['first','last'], value_vars=['height','weight'],
              var_name='body', value_name='data')
```

```
Out[12]:
```

	first	last	body	data
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

예제 데이터2

```
In [13]: index = pd.MultiIndex.from_tuples([('Person','A'),('Person','B')])

df2 = pd.DataFrame({'first':['John','Mary'],
                    'last':['Doe','Bo'],
                    'height':[5.5,6.0],
                    'weight':[130,150]},
                    index=index)

df2
```

```
Out[13]:
```

		first	last	height	weight
Person	A	John	Doe	5.5	130
	B	Mary	Bo	6.0	150

```
In [14]: df2.melt()
```

```
Out[14]:
```

	variable	value
0	first	John
1	first	Mary
2	last	Doe
3	last	Bo
4	height	5.5
5	height	6.0
6	weight	130
7	weight	150

```
In [15]: df2.melt(id_vars=['first','last'])
```

Out[15]:

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

- `melt(id_vars=[ ], ignore_index=True | False )`

```
In [16]: df2.melt(ignore_index=True)
df2.melt(ignore_index=False)
```

Out[16]:

	variable	value
0	first	John
1	first	Mary
2	last	Doe
3	last	Bo
4	height	5.5
5	height	6.0
6	weight	130
7	weight	150

Out[16]:

		variable	value
Person	A	first	John
	B	first	Mary
	A	last	Doe
	B	last	Bo
	A	height	5.5
	B	height	6.0
	A	weight	130
	B	weight	150

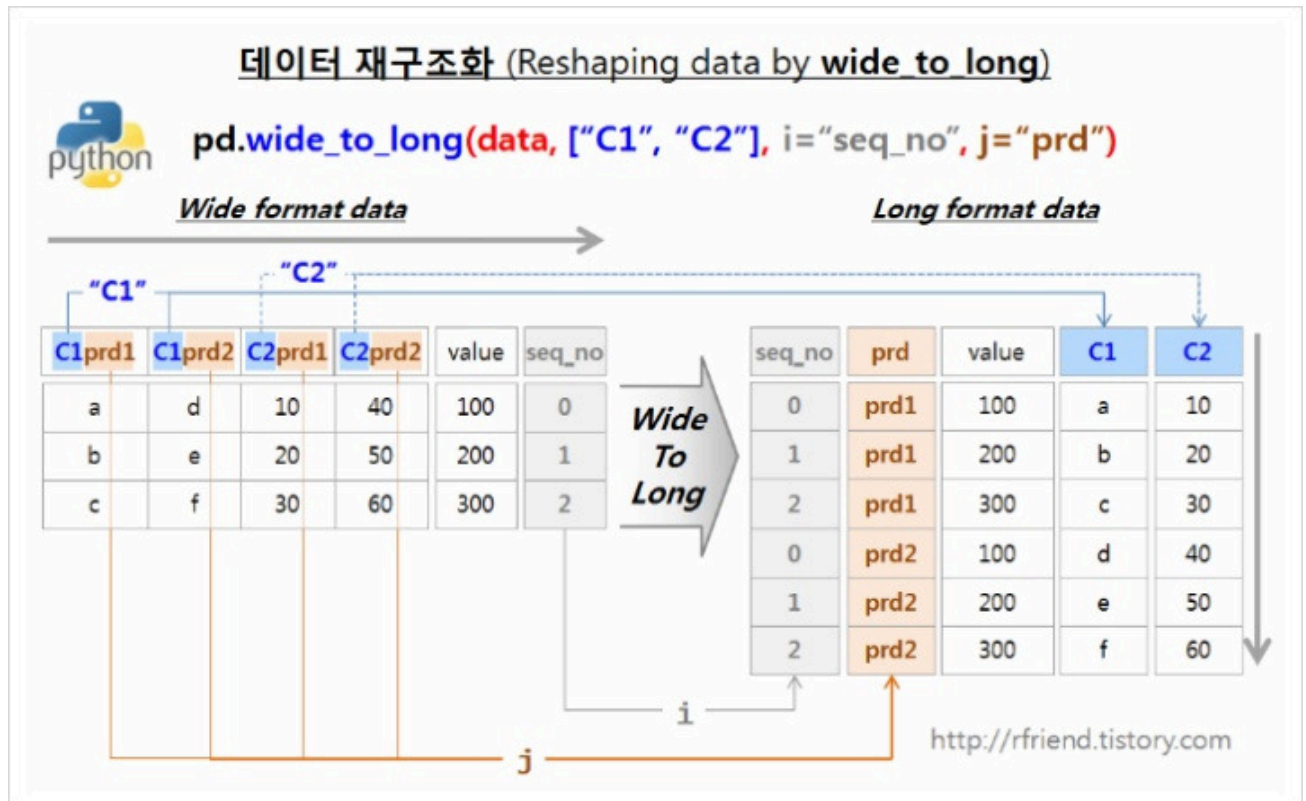
```
In [17]: df2.melt(id_vars=['first','last'], ignore_index=False)
```

Out[17]:

		first	last	variable	value
Person	A	John	Doe	height	5.5
	B	Mary	Bo	height	6.0
	A	John	Doe	weight	130.0
	B	Mary	Bo	weight	150.0

4. Wide-to-long

- melt()와 유사함
- 컬럼 매칭을 위한 사용자 조작이 가능



- 출처 : <https://rfriend.tistory.com/279>

wide_to_long()

`pandas.wide_to_long(df, stubnames, i, j, sep="", suffix='Wd+')`

- df : dataframe
- stubnames : stub 이름(들), wide 포맷 변수들은 stub 이름으로 시작할 것으로 가정함
- i : id 변수(들)로 사용될 컬럼(들)
- j : sub-observation 변수 이름. long 포맷에 suffix 이름
- sep : wide포맷에서 변수 이름 구분을 위해 사용하는 문자
- suffix : 정규표현식

https://pandas.pydata.org/docs/reference/api/pandas.wide_to_long.html#pandas.wide_to_long

예제 데이터1

```
In [18]: np.random.seed(123)
df = pd.DataFrame({'A1970': {0:'a',1:'b',2:'c'},
                  'A1980': {0:'d',1:'e',2:'f'},
                  'B1970': {0:2.5, 1:1.2, 2:0.7},
                  'B1980': {0:3.2, 1:1.3, 2:0.1},
                  'X': dict(zip(range(3), np.random.randn(3)))
                  })

df['id'] = df.index
df
```

```
Out[18]:
```

	A1970	A1980	B1970	B1980	X	id
0	a	d	2.5	3.2	-1.085631	0
1	b	e	1.2	1.3	0.997345	1
2	c	f	0.7	0.1	0.282978	2

```
In [20]: result = pd.wide_to_long(df, stubnames=['A','B'], i='id', j='year')
result
# i='id', j='year' => 행인덱스가 됨(멀티인덱스)
```

```
Out[20]:
```

		X	A	B
id	year			
0	1970	-1.085631	a	2.5
1	1970	0.997345	b	1.2
2	1970	0.282978	c	0.7
0	1980	-1.085631	d	3.2
1	1980	0.997345	e	1.3
2	1980	0.282978	f	0.1

```
In [21]: result.index
```

```
Out[21]: MultiIndex([(0, 1970),
                      (1, 1970),
                      (2, 1970),
                      (0, 1980),
                      (1, 1980),
                      (2, 1980)],
                      names=['id', 'year'])
```

예제 데이터2

```
In [22]: df = pd.DataFrame({
    'fam_id':[1,1,1,2,2,2,3,3,3],
    'birth':[1,2,3]*3,
    'ht1':[2.8, 2.9, 2.2, 2.0, 1.8, 1.9, 2.2, 2.3, 2.1],
    'ht2':[3.4, 3.8, 2.9, 3.2, 2.8, 2.4, 3.3, 3.4, 2.9]
})
df
```


Out[22]:

	fam_id	birth	ht1	ht2
0	1	1	2.8	3.4
1	1	2	2.9	3.8
2	1	3	2.2	2.9
3	2	1	2.0	3.2
4	2	2	1.8	2.8
5	2	3	1.9	2.4
6	3	1	2.2	3.3
7	3	2	2.3	3.4
8	3	3	2.1	2.9

In [24]:

```
result = pd.wide_to_long(df, stubnames='ht',
                        i=['fam_id','birth'], j='age')
result
```

Out[24]:

			ht	
fam_id	birth	age		
1	1	1	2.8	
		2	3.4	
	2	1	2.9	
		2	3.8	
	3	1	2.2	
		2	2.9	
2	1	1	2.0	
		2	3.2	
	2	1	1.8	
		2	2.8	
	3	1	1.9	
		2	2.4	
3	1	1	2.2	
		2	3.3	
	2	1	2.3	
		2	3.4	
	3	1	2.1	
		2	2.9	

In [26]:

```
result.unstack()
result.unstack().reset_index()
```

Out[26]:

		ht		
		age	1	2
fam_id	birth			
1	1	2.8	3.4	
	2	2.9	3.8	
	3	2.2	2.9	
2	1	2.0	3.2	
	2	1.8	2.8	
	3	1.9	2.4	
3	1	2.2	3.3	
	2	2.3	3.4	
	3	2.1	2.9	

Out[26]:

		fam_id	birth	ht	
		age	1	2	
0	1	1	2.8	3.4	
1	1	2	2.9	3.8	
2	1	3	2.2	2.9	
3	2	1	2.0	3.2	
4	2	2	1.8	2.8	
5	2	3	1.9	2.4	
6	3	1	2.2	3.3	
7	3	2	2.3	3.4	
8	3	3	2.1	2.9	

```
In [29]: # 와이드-투-롱으로 변경된 데이터를 다시 원데이터 구조로 변경
org = result.unstack()
org.columns
coln = org.columns.map('{0[0]}{0[1]}'.format)
coln
org.columns = coln
org.reset_index()
```

Out[29]: MultiIndex([('ht', 1),
 ('ht', 2)],
 names=[None, 'age'])

Out[29]: Index(['ht1', 'ht2'], dtype='str')

Out[29]:

	fam_id	birth	ht1	ht2
0	1	1	2.8	3.4
1	1	2	2.9	3.8
2	1	3	2.2	2.9
3	2	1	2.0	3.2
4	2	2	1.8	2.8
5	2	3	1.9	2.4
6	3	1	2.2	3.3
7	3	2	2.3	3.4
8	3	3	2.1	2.9

예제 데이터3

```
In [27]: np.random.seed(3)
df = pd.DataFrame({'A(weekley)-2010': np.random.rand(3),
                  'A(weekley)-2011': np.random.rand(3),
                  'B(weekley)-2010': np.random.rand(3),
                  'B(weekley)-2011': np.random.rand(3),
                  'X' : np.random.randint(3, size=3)})
df['id'] = df.index
df
```

Out[27]:

	A(weekley)-2010	A(weekley)-2011	B(weekley)-2010	B(weekley)-2011	X	id
0	0.550798	0.510828	0.125585	0.440810	0	0
1	0.708148	0.892947	0.207243	0.029876	2	1
2	0.290905	0.896293	0.051467	0.456833	1	2

```
In [31]: pd.wide_to_long(df, stubnames=['A(weekley)', 'B(weekley)'],
                        i='id', j='year', sep='-')
```

Out[31]:

	X	A(weekley)	B(weekley)
id year			
0 2010	0	0.550798	0.125585
1 2010	2	0.708148	0.207243
2 2010	1	0.290905	0.051467
0 2011	0	0.510828	0.440810
1 2011	2	0.892947	0.029876
2 2011	1	0.896293	0.456833

예제 데이터4. suffixes로 정수를 갖지 않는 경우

```
In [32]: df = pd.DataFrame({'id':[1,2],
                          'name':['Alice','Bob'],
                          'math_score':[90,70],
```

```
'eng_score':[80,100] })
```

df

```
Out[32]:
```

	id	name	math_score	eng_score
0	1	Alice	90	80
1	2	Bob	70	100

```
In [34]: pd.wide_to_long(df, stubnames=['math','eng'], i='name', j='subject',  
                        sep='_', suffix='score')
```

```
Out[34]:
```

	id	math	eng		
	name	subject			
	Alice	score	1	90	80
	Bob	score	2	70	100

문제. 다음의 데이터프레임을 wide-to-long을 적용하여 구조 변경하기

```
df = pd.DataFrame({'id':[1]*3+[2]*3+[3]*3,  
                  'birth':[1,2,3]*3,  
                  'score_one':[2.8, 3.1, 4.2, 3.1, 1.8, 1.9, 2.2, 2.3, 2.1],  
                  'score_two':[3.4, 5.1, 4.3, 4.5, 4.7, 4.8, 4.2, 3.3, 3.9]  
                  })
```

```
In [35]: df = pd.DataFrame({'id':[1]*3+[2]*3+[3]*3,  
                          'birth':[1,2,3]*3,  
                          'score_one':[2.8, 3.1, 4.2, 3.1, 1.8, 1.9, 2.2, 2.3, 2.1],  
                          'score_two':[3.4, 5.1, 4.3, 4.5, 4.7, 4.8, 4.2, 3.3, 3.9]  
                          })  
df
```

```
Out[35]:
```

	id	birth	score_one	score_two
0	1	1	2.8	3.4
1	1	2	3.1	5.1
2	1	3	4.2	4.3
3	2	1	3.1	4.5
4	2	2	1.8	4.7
5	2	3	1.9	4.8
6	3	1	2.2	4.2
7	3	2	2.3	3.3
8	3	3	2.1	3.9

```
In [39]: pd.wide_to_long(df, stubnames='score', i=['id','birth'],  
                        j='nth', sep='_', suffix='\\w+')  
# 정규표현식 : '\\w+' => 문자 0~9a~zA~Z를 의미, r'\\w+' => r:raw
```

Out[39]:

			score
id	birth	nth	
1	1	one	2.8
		two	3.4
	2	one	3.1
		two	5.1
	3	one	4.2
		two	4.3
2	1	one	3.1
		two	4.5
	2	one	1.8
		two	4.7
	3	one	1.9
		two	4.8
3	1	one	2.2
		two	4.2
	2	one	2.3
		two	3.3
	3	one	2.1
		two	3.9

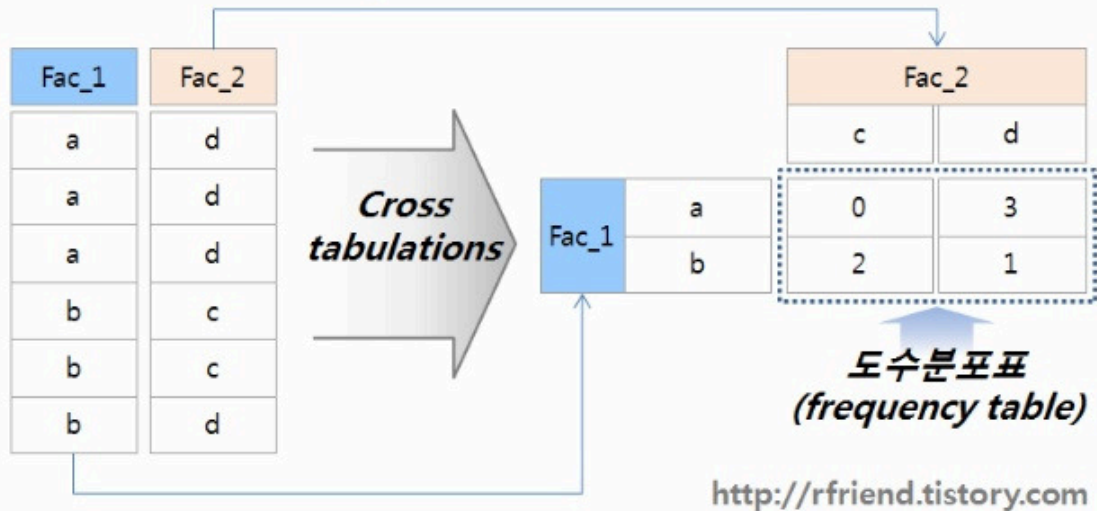
5. 교차표 crosstab

- 2개 이상의 요인을 위한 교차표(cross tabulation) 계산

데이터 재구조화 (Reshaping data by cross tabulations)



pd.crosstab(**index**, # values to group by in the rows
columns, # values to group by in the columns
rownames, # assigning row names
colnames, # assigning column names
margins, # adding row/column margins
normalize) # Normalizing by dividing all values by the sum of values



crosstab()

```
pandas.crosstab(index, columns, values=None, rownames=None, colnames=None,  
                 aggfunc=None, margins=False, margins_name='All',  
                 dropna=True, normalize=False)
```

- index : 행
- columns : 열
- values : factor에 의해 집계할 값들
- aggfunc : 계산 함수
- margins : 행/열 소계
- dropna : 모두 결측치를 갖는 컬럼은 포함하지 않음

<https://pandas.pydata.org/docs/reference/api/pandas.crosstab.html>

예제 데이터1. 일차원 데이터들을 이용한 교차표

```
In [40]: a = np.array(['foo']*4+['bar']*4+['foo']*3, dtype=object)  
b = np.array(['one']*3+['two'], 'one']*2+['two']*3+['one'], dtype=object)  
c = np.array(['dull', 'dull', 'shiny', 'dull', 'dull', 'shiny',  
             'dull', 'shiny', 'dull', 'shiny', 'shiny'], dtype=object)  
a  
b  
c
```

```
Out[40]: array(['foo', 'foo', 'foo', 'foo', 'bar', 'bar', 'bar', 'bar', 'foo',  
              'foo', 'foo'], dtype=object)
```

```
Out[40]: array(['one', 'one', 'one', 'two', 'one', 'two', 'one', 'two', 'two',  
              'two', 'one'], dtype=object)
```

```
Out[40]: array(['dull', 'dull', 'shiny', 'dull', 'dull', 'shiny', 'dull', 'shiny',  
              'dull', 'shiny', 'shiny'], dtype=object)
```

- pd.crosstab()

```
In [42]: pd.crosstab(a,b)
pd.crosstab(a,[b,c])
```

```
Out[42]:
```

row_0		
col_0	one	two
bar	2	2
foo	4	3

```
Out[42]:
```

row_0				
col_0	one		two	
col_1	dull	shiny	dull	shiny
bar	2	0	0	2
foo	2	2	2	1

- `pd.crosstab(index=, columns=)`

```
In [43]: pd.crosstab(index=a, columns=b)
pd.crosstab(index=a, columns=[b,c])
```

```
Out[43]:
```

row_0		
col_0	one	two
bar	2	2
foo	4	3

```
Out[43]:
```

row_0				
col_0	one		two	
col_1	dull	shiny	dull	shiny
bar	2	0	0	2
foo	2	2	2	1

- `pd.crosstab(, rownames=, colnames=, )`

```
In [44]: pd.crosstab(index=a, columns=[b,c], rownames=['a'], colnames=['b', 'c'])
```

```
Out[44]:
```

a				
b	one		two	
c	dull	shiny	dull	shiny
bar	2	0	0	2
foo	2	2	2	1

예제 데이터2. 범주형(Categorical) 데이터를 이용한 교차표 생성

```
In [45]: A = pd.Categorical(['a', 'b', 'b', 'a'], categories=['a', 'b', 'c'])
B = pd.Categorical(['d', 'd', 'e', 'd'], categories=['d', 'e', 'f'])
```

A
B

```
Out[45]: ['a', 'b', 'b', 'a']  
Categories (3, str): ['a', 'b', 'c']
```

```
Out[45]: ['d', 'd', 'e', 'd']  
Categories (3, str): ['d', 'e', 'f']
```

- `pd.crosstab()`

```
In [46]: pd.crosstab(A,B)
```

```
Out[46]: col_0  d  e  
row_0  
a    2  0  
b    1  1
```

- `pd.crosstab(, dropna=False)`

```
In [47]: pd.crosstab(A,B, dropna=False)
```

```
Out[47]: col_0  d  e  f  
row_0  
a    2  0  0  
b    1  1  0  
c    0  0  0
```

예제 데이터3. 데이터 프레임을 사용한 교차표 생성

```
In [49]: df = pd.DataFrame({'A':[1,2,2,2,2],  
                           'B':[3,3,4,4,4],  
                           'C':[1,1,np.nan,1,1]})  
df
```

```
Out[49]:
```

	A	B	C
0	1	3	1.0
1	2	3	1.0
2	2	4	NaN
3	2	4	1.0
4	2	4	1.0

- `pd.crosstab()`

```
In [52]: pd.crosstab(df.A, df.B)  
pd.crosstab(index=df.A, columns=df.B)  
pd.crosstab(df.A, df.C)  
pd.crosstab(df.A, df.C, dropna=False)
```


Out[52]: **B 3 4**

A

1 1 0

2 1 3

Out[52]: **B 3 4**

A

1 1 0

2 1 3

Out[52]: **C 1.0**

A

1 1

2 3

Out[52]: **C 1.0 NaN**

A

1 1 0

2 3 1

- `pd.crosstab(, normalize=True)`

```
In [53]: pd.crosstab(index=df.A, columns=df.B, normalize=True)
```

Out[53]: **B 3 4**

A

1 0.2 0.0

2 0.2 0.6

- `pd.crosstab(values=, aggfunc=)`

```
In [57]: df
pd.crosstab(index=df.A, columns=df.B, values=df.C, aggfunc='sum')
# values 속성은 반드시 aggfunc 과 같이 지정해 사용해야 함
```

Out[57]:

A B C

0 1 3 1.0

1 2 3 1.0

2 2 4 NaN

3 2 4 1.0

4 2 4 1.0

Out[57]:

	B	3	4
A			
1	1.0	NaN	
2	1.0	2.0	

- `pd.crosstab(margins=)`

```
In [60]: pd.crosstab(index=df.A, columns=df.B, margins=True)
pd.crosstab(index=df.A, columns=df.B, normalize=True,
            margins=True)
pd.crosstab(index=df.A, columns=df.B, normalize=True,
            margins=True, margins_name='Total')
```

Out[60]:

	B	3	4	All
A				
1	1	0	1	
2	1	3	4	
All	2	3	5	

Out[60]:

	B	3	4	All
A				
1	0.2	0.0	0.2	
2	0.2	0.6	0.8	
All	0.4	0.6	1.0	

Out[60]:

	B	3	4	Total
A				
1	0.2	0.0	0.2	
2	0.2	0.6	0.8	
Total	0.4	0.6	1.0	

6. explode

- 데이터프레임의 컬럼 내에 리스트와 같은 값들을 갖는 경우 컬럼의 리스트 요소들을 행으로 분리
- 리스트와 같은 요소를 시리즈의 값으로 갖는 경우 행으로 분리

explode()

- `Series.explode(ignore_index=False)`
- `DataFrame.explode(column, ignore_index=False)`
- <https://pandas.pydata.org/docs/reference/api/pandas.Series.explode.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.explode.htm>

예제 데이터1. 컬럼의 값으로 리스트를 갖는 데이터프레임

```
In [61]: keys = ['panda1', 'panda2', 'panda3']
values = [['eats', 'shoots'], ['shoots', 'leaves'], ['eats', 'leaves']]
df = pd.DataFrame({'keys':keys,
                    'values':values})
df
```

```
Out[61]:
```

	keys	values
0	panda1	[eats, shoots]
1	panda2	[shoots, leaves]
2	panda3	[eats, leaves]

- 리스트를 값으로 갖는 컬럼을 여러 컬럼으로 분할하여 구성 : apply() 함수 사용

```
In [64]: df[['act1', 'act2']] = df['values'].apply(pd.Series)
df
```

```
Out[64]:
```

	keys	values	act1	act2
0	panda1	[eats, shoots]	eats	shoots
1	panda2	[shoots, leaves]	shoots	leaves
2	panda3	[eats, leaves]	eats	leaves

- explode() 함수 사용하여 리스트의 값을 하나의 컬럼의 값으로 구성

```
In [66]: df.drop(columns=['act1', 'act2'], inplace=True)
df
df.explode('values')
```

```
Out[66]:
```

	keys	values
0	panda1	[eats, shoots]
1	panda2	[shoots, leaves]
2	panda3	[eats, leaves]

```
Out[66]:
```

	keys	values
0	panda1	eats
0	panda1	shoots
1	panda2	shoots
1	panda2	leaves
2	panda3	eats
2	panda3	leaves

예제 데이터2. 시리즈의 행 요소가 리스트를 포함하는 경우

```
In [68]: s = pd.Series([[1,2,3], 'foo', [], ['a', 'b']])
s
```

```
Out[68]: 0    [1, 2, 3]
          1         foo
          2          []
          3    [a, b]
          dtype: object
```

```
In [69]: s.explode()
```

```
Out[69]: 0      1
          0      2
          0      3
          1    foo
          2    NaN
          3     a
          3     b
          dtype: object
```

예제 데이터3. 콤마로 구분된 문자열을 값으로 갖는 데이터프레임

```
In [76]: df = pd.DataFrame([
          {'var1': 'a,b,c', 'var2': 1},
          {'var1': 'd,e,f', 'var2': 2}])
df
```

```
Out[76]:   var1  var2
0  a,b,c    1
1  d,e,f    2
```

```
In [77]: df.var1.str.split(',')
df.var1.str.split(',', expand=True)
```

```
Out[77]: 0    [a, b, c]
          1    [d, e, f]
          Name: var1, dtype: object
```

```
Out[77]:   0  1  2
          0  a  b  c
          1  d  e  f
```

```
In [78]: df.var1 = df.var1.str.split(',')
df
df.explode('var1')
```

```
Out[78]:   var1  var2
0  [a, b, c]    1
1  [d, e, f]    2
```

```
Out[78]:
```

	var1	var2
0	a	1
0	b	1
0	c	1
1	d	2
1	e	2
1	f	2

참고. df.assign()

- 데이터프레임에 새로운 컬럼을 할당
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.assign.html>

```
In [79]: df = pd.DataFrame([
        {'var1': 'a,b,c', 'var2': 1},
        {'var1': 'd,e,f', 'var2': 2}])
df
```

```
Out[79]:
```

	var1	var2
0	a,b,c	1
1	d,e,f	2

```
In [81]: # df.var1 = df.var1.str.split(',')
df.assign(var1=df.var1.str.split(','))
```

```
Out[81]:
```

	var1	var2
0	[a, b, c]	1
1	[d, e, f]	2

[데이터 재구조화 정리]

- pivot : 돌려서 가로로 펼치기(행, 열의 요소가 유일한 경우)
- pivot_table : pivot + 집계
- crosstab : 교차빈도표 생성
- stack : 인덱스 레벨 추가함으로써 아래로 쌓기
- unstack : 행인덱스를 옆으로 펼치기(컬럼 인덱스로 변경)
- melt : 녹여서 세로로 모으기
- wide_to_long : 넓은 반복 컬럼을 긴 구조로 바꾸기(컬럼들의 이름이 그룹화가 가능한 경우)
- explode : 한 셀을 터뜨려 여러 행으로 만들기

함수	방향	주요 목적	집계 기능	중복 허용	주요 용도
pivot	long → wide	행 값을 컬럼으로 변환	없음	불가	구조 재배열
pivot_table	long → wide	pivot + 집계	가능	가능	요약 통계표
crosstab	long → wide	교차 빈도표 생성	가능	가능	범주형 빈도 분석
melt	wide → long	여러 컬럼을 행으로 변환	없음	가능	tidy data
stack	columns → index	컬럼 레벨을 인덱스로 이동	없음	가능	MultilIndex 처리

함수	방향	주요 목적	집계 기능	중복 허용	주요 용도
unstack	index → columns	인덱스 레벨을 컬럼으로 이동	없음	가능	그룹 결과 재구성
explode	list → rows	리스트 셀을 행으로 분해	없음	가능	다중값 컬럼 처리
wide_to_long	wide → long	패턴 컬럼명을 long 변환	없음	가능	반복 측정 데이터
